



J.A.R.V.I.S.

Personal AI Assistant with Holographic Gesture HUD

Project Specification Sheet | Version 1.1 | October 2026

Owner: Lyndon Jugalbot

1. Project Overview

JARVIS is a personal AI assistant you can talk to and control with hand gestures. It listens for a wake word, understands requests using a large language model, takes real actions through tools (calendar, email, files, web, system), remembers context over time, and shows everything on a sci-fi holographic-style HUD in the browser, which you manipulate with your hands through a normal webcam. Its brain runs on the Claude and ChatGPT subscriptions you already pay for, then on free services, and only as a capped last resort on a pay-per-use API.

1.1 Goals

- A working voice + text assistant that completes useful daily tasks, not just chat.
- A holographic HUD with panels you can grab, move, maximize, minimize and dismiss with hand gestures.
- Modular design: adding a new tool, gesture or model provider should take minutes, not a refactor.
- Runs on a normal PC or laptop with a webcam; no special hardware required.
- Subscription-first: answers come from your Claude and ChatGPT plans and free services. Pay-per-use API calls happen only when all of those are at their limits, and never above a monthly cap.

1.2 Non-goals (v1)

- True volumetric holograms or AR headset support (screen-based effect only).
- Multi-user accounts, cloud hosting or a mobile app.
- Sharing subscription access: JARVIS uses your own logins, for your own use only.
- Fully autonomous actions without confirmation (sending, deleting and paying always ask first).

1.3 Success criteria

- Say "Hey Jarvis, what's on my calendar tomorrow?" and get a spoken answer plus a calendar panel on the HUD.
- Pinch-drag, fist-minimize and two-hand spread-maximize work reliably (95%+ in good lighting) with no accidental triggers during normal movement, including the look-alike pairs in section 7.1.
- Simple voice requests answer in under about 3 seconds end-to-end on the primary (Claude plan) path.
- A normal month costs nothing beyond your existing subscriptions. When they run out, JARVIS keeps working and paid spend stops at the cap.

2. System Architecture

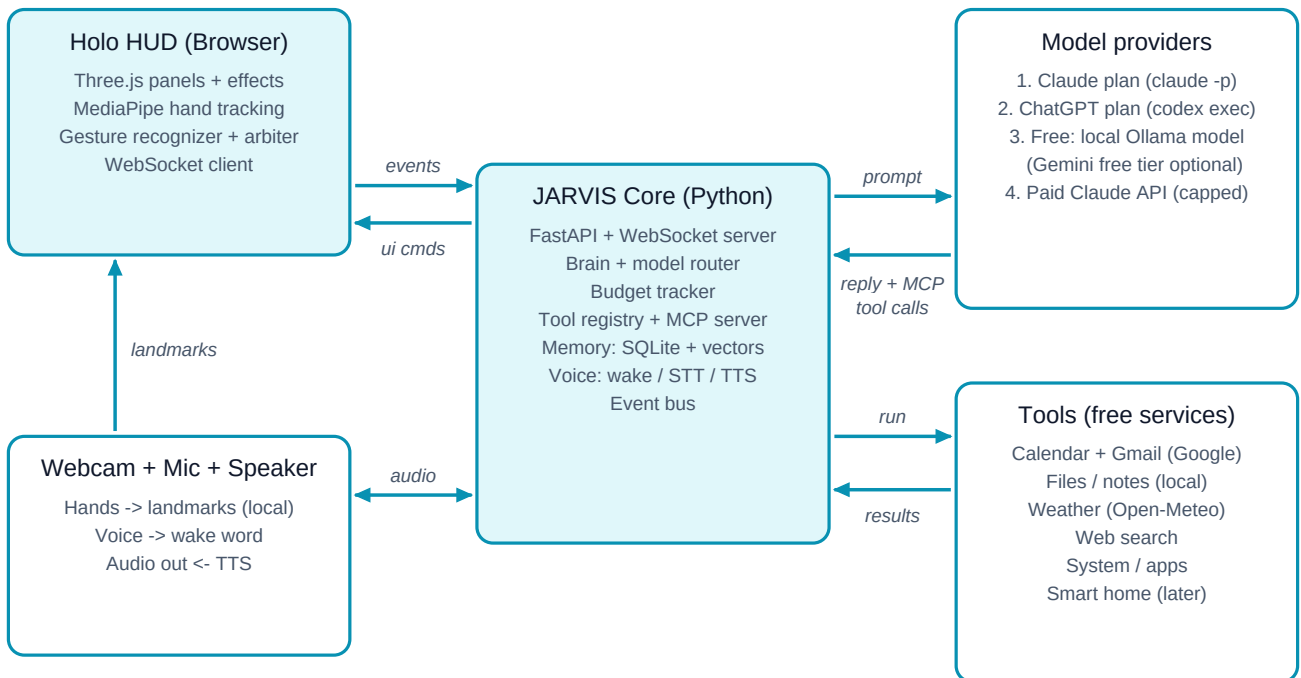
Two processes talk over a local WebSocket: a Python core (brain, model router, tools, memory, voice) and a browser HUD (rendering, hand tracking, gestures). Hand tracking runs entirely in the browser so gestures stay fast and no video ever leaves the machine. The core reaches a language model through a router that tries your subscriptions first. Subscription models run inside their official command-line tools and call JARVIS's tools back over MCP (Model Context Protocol), so every provider uses the same tools and the same safety checks.

2.1 Request flow (voice example)

1. Wake word detected locally -> mic stream recorded until silence.
2. Speech-to-text converts audio to text.
3. The brain builds the request: system prompt + recent turns + the top 5 relevant memories.
4. The router sends it to the first available provider: Claude plan -> ChatGPT plan -> free local model -> paid API.
5. The model calls tools (e.g. `get_events`). The core runs them through one registry, whichever provider asked; risky tools wait for your confirmation.

- The final reply is spoken by TTS, the core tells the HUD to open a panel with the data, and the status bar shows which provider answered.
- If the provider hits its limit mid-request, the router retries the same turn on the next provider. Tools that already ran are not run again.

2.2 Component diagram



3. Model Routing & Cost Policy

JARVIS treats the language model as a replaceable part. A router holds an ordered list of providers and always uses the first one that is available. Your subscriptions come first because they cost nothing extra, free options come next, and the pay-per-use API is the last resort and is capped. The order lives in `config.toml`.

3.1 Provider chain

#	Provider	How JARVIS uses it	Cost	Limit signal
1	Claude plan (Pro / Max)	Official Claude Code CLI in headless mode: <code>claude -p</code> , kept warm with stream-json input and output. JARVIS tools come in through <code>--mcp-config</code> with <code>--strict-mcp-config</code> ; built-in tools are limited with <code>--tools</code> to web search and web fetch.	Included in your plan	Usage-limit error from the CLI (5-hour and weekly windows); reset time read from the message when present
2	ChatGPT plan (Plus / Pro)	Official Codex CLI: <code>codex exec --json</code> with a dedicated <code>jarvis</code> profile that registers the JARVIS MCP server. Shell tool disabled, read-only sandbox, empty working folder, <code>--ephemeral</code> , <code>--skip-git-repo-check</code> , live web search on.	Included in your plan	<code>turn.failed</code> or error event reporting a usage limit
3	Free: local model	Ollama running a tool-calling model on your machine, driven through Codex's <code>--oss</code> local-provider mode so it gets the same MCP tools.	\$0; private; works offline	Ollama not running or model missing

#	Provider	How JARVIS uses it	Cost	Limit signal
3b	Free: Gemini API free tier (optional)	Off by default: free-tier prompts may be used to improve Google's products, and JARVIS prompts carry personal context. Turn on only for requests without email, file or memory content.	\$0, rate-limited	HTTP 429
4	Paid API (last resort)	Claude API through the official anthropic Python SDK with tool use. Default model <code>claude-opus-5-5</code> (\$4 / \$20 per million input / output tokens, Oct 2026); the model name lives in config.	Pay per token, capped	Monthly cap reached; HTTP 429

3.2 Failover rules

- A provider is skipped when it is not installed, not signed in, or cooling down.
- When a provider reports a usage limit, it cools down until the reset time it gives. With no reset time, the router re-checks a subscription provider every 15 minutes. The paid API is never probed.
- Each switch shows on the HUD status bar and is announced once ("Claude's limit is reached, switching to ChatGPT").
- JARVIS owns the conversation history and memory and sends the needed context with every turn, so switching providers mid-conversation loses nothing.
- Every tool call in a turn is logged with its arguments and result. If a turn moves to another provider, finished calls are answered from that log instead of running again, and a risky tool never runs twice without a fresh confirmation.
- A provider that fails for any other reason (crash, no output for 20 s, unreadable output) is retried once, then skipped for that turn.

3.3 Paid fallback and budget

- `paid_fallback` setting: auto (default: use the paid API when everything above is unavailable), ask (JARVIS asks by voice or HUD before the first paid call each day) or off.
- `paid_monthly_cap_usd` (default 10). Spend is computed from the token usage in every API response and stored in SQLite. JARVIS warns at 80%; at 100% the paid tier switches off until next month and JARVIS says when the next free option resets.
- Set the same amount as a spend limit in the Anthropic Console as a hard backstop.
- An amber PAID badge shows on the HUD whenever the paid tier answers.

3.4 Free services for tools

Need	Free choice	Notes
Weather	Open-Meteo	No API key or account needed.
Web search	The built-in web search of the subscription CLI that is answering	Included in your plan.
Web search (fallback)	Tavily free plan (1,000 credits a month, no card) or DuckDuckGo via the <code>ddgs</code> library	Used only when the answering provider has no search of its own.
Calendar and email	Google Calendar and Gmail APIs	Free within Google's quotas.
Speech, wake word, voice	faster-whisper, openWakeWord, Piper	Run locally.
Memory embeddings	Local embedding model (fastembed)	Keeps memory private and free.

3.5 Account settings and terms

- JARVIS only calls the official CLIs, signed in as you. It never reads, copies or reuses their login tokens. Anthropic does not allow third-party products to offer claude.ai login to other people, so JARVIS stays a personal tool.
- As of October 2026, Anthropic says c laude -p and Agent SDK use draw from your normal plan usage limits. A separate monthly Agent SDK credit announced for June 2026 was paused. Re-check when Anthropic publishes its update, and give the Claude provider its own budget if that credit returns.
- Turn off any pay-as-you-go overage or automatic credit top-up on both plans, so hitting a plan limit is a clean stop that JARVIS detects instead of a silent charge.
- Free cloud tiers that may train on prompts never receive email, file or memory content.

4. Tech Stack

Layer	Choice	Why / notes
Core server	Python 3.11+, FastAPI, Uvicorn, websockets	Python is your strongest language; FastAPI gives HTTP + WebSocket in one.
Brain (primary)	Claude Code CLI, headless (c laude -p), on your Claude plan	Included in your plan; strong tool calling.
Brain (second)	Codex CLI (codex exec), on your ChatGPT plan	Included in your plan; same tools over MCP.
Brain (free)	Ollama local model; Gemini API free tier optional	\$0. Weaker tool use, so it comes after the subscriptions.
Brain (paid)	Claude API via the anthropic SDK, c laude-opus-5-5	Last resort, capped. Model name lives in config.
Tool bridge	Official MCP Python SDK (FastMCP)	One tool registry serves every provider.
Speech-to-text	faster-whisper (local), small or base model	Free, private, fast on CPU; move to medium if accuracy is low.
Wake word	openWakeWord (free)	Ships a "hey jarvis" model.
Text-to-speech	Piper (local, free); ElevenLabs optional	ElevenLabs is paid, so it stays off unless you turn it on.
Memory	SQLite + sqlite-vec; fastembed for embeddings	Facts and preferences in tables, long-term recall via local embeddings.
Frontend	Vanilla JS + Vite, Three.js	No framework needed; Vite gives fast dev reload.
Hand tracking	MediaPipe Tasks Vision - HandLandmarker	21 landmarks per hand, ~30 fps in browser, runs locally.
Effects	Three.js EffectComposer + UnrealBloomPass, custom shaders	Glow, scanlines, flicker, holographic transparency.
Integrations	Google Calendar / Gmail APIs, Open-Meteo, Tavily or ddgs, httpx, psutil	All free at personal volumes.
Tooling	uv, pytest, ruff, Git + GitHub	Keep it clean from day one.

5. Backend Specification (Python Core)

5.1 Modules

Module	Responsibility
main.py	Starts FastAPI and the WebSocket endpoint /ws, loads config, wires modules together.
brain/router.py	Provider chain: availability, cooldowns, failover, and retrying a turn on the next provider without re-running finished tool calls.
brain/providers/claude_cli.py	Runs a warm <code>claude -p stream-json</code> process, reads its events, detects usage-limit errors.
brain/providers/codex_cli.py	Runs <code>codex exec --json</code> with the <code>jarvis</code> profile (and <code>--oss</code> for the local model), reads JSONL events, detects limits.
brain/providers/gemini_free.py	Optional Gemini free-tier adapter with native tool calling.
brain/providers/anthropic_api.py	Paid Claude API with the tool-use loop; reports token usage to the budget.
brain/budget.py	Tracks paid spend per month, enforces the cap and the <code>paid_fallback</code> mode.
brain/prompt.py	System prompt: JARVIS personality, rules (confirm before risky actions), current date/time, user profile.
tools/registry.py	Decorator <code>@tool</code> that registers a function plus its JSON schema automatically. One registry for every provider.
tools/mcp_server.py	Serves the registry over MCP on 127.0.0.1 for the CLI providers. The confirmation gate runs here.
tools/*.py	One file per tool group: calendar, email, files, web, weather, system, notes.
auth/google.py	Google sign-in, token refresh and the reconnect flow (section 5.5).
memory/store.py	Save/retrieve facts, conversation summaries and embeddings; returns top-k relevant memories.
voice/wake.py, stt.py, tts.py	Wake word listener, transcription, speech output. Each can be switched off in config.
events.py	Simple async event bus so voice, brain and HUD stay decoupled.

5.2 Tool interface and confirmation

Every tool is a plain async Python function with type hints and a docstring. The registry turns it into a tool definition for the paid API and an MCP tool for the CLI providers.

```
@tool(confirm=False)
async def get_events(day: str = "today") -> list[dict]:
    """Return calendar events for a day ("today", "tomorrow" or YYYY-MM-DD)."""
    ...

@tool(confirm=True) # HUD/voice must get a "yes" before this runs
async def send_email(to: str, subject: str, body: str) -> str:
    ...
```

Confirmation is enforced by JARVIS's tool runner, not by the model or the CLI. A `confirm=True` tool sends `confirm_request` to the HUD and waits up to 30 seconds for a thumbs-up or a spoken "yes"; silence counts as no. Because the gate lives inside JARVIS, it works the same whichever provider made the call. Each CLI's MCP tool timeout is set above 30 seconds (`Codex tool_timeout_sec`, `Claude Code MCP_TOOL_TIMEOUT`) so the wait is never cut short.

5.3 v1 tool list

Tool	Purpose	Service (cost)	Confirm?
get_time / get_weather	Date, time, local weather	Local clock; Open-Meteo (free)	No
get_events, create_event	Read and add calendar events	Google Calendar API (free)	Create: yes
search_email, draft_email, send_email	Inbox search, drafting, sending	Gmail API (free)	Send: yes
search_files, read_file, write_note	Find files, read text, save notes	Local	Write: yes
web_search	Look up current information. Only offered to providers without their own search.	Tavily free plan or ddgs (free)	No
open_app, open_url, system_stats	Launch apps, open sites, CPU/RAM/battery	Local (psutil)	No
remember, recall, forget	Store, fetch and delete personal facts	Local SQLite	Forget: yes
show_panel	Ask the HUD to display data (text, list, chart, image)	Local	No

5.4 Memory design

- facts table: key, value, source, updated_at (e.g. preferred_name = Lyndon).
- messages table: rolling history; last 20 turns sent to the model, older turns summarized. JARVIS owns this history, so it survives a switch between providers.
- embeddings: summaries and notes embedded locally with fastembed for semantic recall; top 5 matches injected into the prompt.
- A "forget" command deletes by key or topic.

5.5 Google sign-in (Calendar and Gmail)

- Create your own Google Cloud project with an OAuth client of type **Desktop app** and user type **External** (personal Gmail accounts can't use Internal).
- Set the consent screen's publishing status to **In production**. While an app is in Testing, Google expires its refresh tokens after 7 days, which would force a sign-in every week.
- Google's app verification isn't needed for personal use. The first sign-in shows a "Google hasn't verified this app" warning, which you accept once (Advanced -> Go to JARVIS).
- Request only the scopes needed: `calendar.events`, `gmail.readonly`, `gmail.compose`.
- Tokens are stored in `~/jarvis/google_token.json` with owner-only permissions, never in the repo.
- If a refresh still fails (access revoked, password changed, or 6 months unused), JARVIS says "I've lost access to Google" and the HUD shows a Reconnect panel that reruns the browser sign-in. Google tools return a friendly error until then.

6. Holographic HUD Specification

6.1 Layout

- Fullscreen dark canvas (near-black #05080F) with a faint animated grid and scanlines.
- Core orb center-bottom: pulsing rings that react to listening, thinking and speaking states.
- Panels: floating translucent cards in 3D space, slightly tilted, with depth parallax as the hand moves.

- Transcript strip along the bottom showing what you said and JARVIS's reply.
- Status bar top: time, gesture-armed indicator, mic state, connection state, and the provider answering right now (amber PAID badge when it is the paid API).
- Fingertip cursor: glowing dot with a trailing ring; fills up to show pinch strength.

6.2 Visual style

Element	Spec
Primary color	Cyan #22D3EE; accents #38BDF8; warning #F59E0B (also the PAID badge); danger #F43F5E
Panels	Background rgba(34,211,238,0.08), 1px cyan border, corner brackets, bloom glow
Typography	Orbitron or Rajdhani for headings, Inter for body text
Effects	UnrealBloom (strength ~1.2), scanline shader, 2-3 frame flicker on open, hex-grid noise
Animation	Panels scale 0.8->1 with opacity fade over 250 ms; minimize collapses into a dock icon
Sound (optional)	Short UI blips on grab, open and close

6.3 Panel object model

```

Panel {
  id: string, type: "text" | "list" | "calendar" | "chart" | "image" | "web",
  title: string, data: any,
  state: "normal" | "focused" | "maximized" | "minimized",
  position: {x, y, z}, size: {w, h}, createdAt: number
}

```

6.4 Frontend modules

File	Responsibility
hud/scene.js	Three.js scene, camera, renderer, post-processing pipeline
hud/panel.js	Panel class: create, render content (via CanvasTexture or CSS3DRenderer), state transitions
hud/effects.js	Bloom, scanline shader, flicker, orb animation
hud/statusbar.js	Time, armed, mic, connection and provider indicators
gestures/tracker.js	Webcam setup, MediaPipe HandLandmarker, smoothing of landmarks
gestures/recognizer.js	Turns landmarks into candidate gestures with thresholds, hold times and cooldowns
gestures/arbiter.js	Applies the rules in section 7.1 so only one gesture fires at a time
gestures/controller.js	Maps gesture events to panel actions (hit-testing via raycaster)
net/socket.js	WebSocket client: first-message auth, auto-reconnect, message routing

7. Gesture Specification

Landmarks are normalized to hand size **h** (wrist to middle-finger knuckle distance) so gestures work at any distance from the camera; speeds are in h per second. All gestures are smoothed with a One Euro filter and require the system to be armed. Every number below is a starting value you can tune live in the settings panel.

Gesture	Detection rule	Action	Hold / cooldown
Arm (open palm)	All 5 fingers extended, palm facing camera, wrist nearly still (< 0.5 h/s)	Enable gestures; auto-disarm after 6 s of no hand	Hold 500 ms; only while disarmed
Point	Index extended, other fingers curled	Move cursor, hover highlights panel	Live
Pinch tap	Thumb-index distance < 0.25 h, released < 300 ms	Click / select panel or button	Cooldown 250 ms
Pinch hold + move	Pinch held > 300 ms with no pinch from the other hand (7.1)	Grab and drag panel	Release on unpinch
Fist	Index-to-pinky tips below their middle joints and thumb tucked (tip within 0.4 h of the index knuckle)	Minimize focused panel to dock	Hold 400 ms, cooldown 800 ms
Two-hand spread	Both hands pinching, distance grows $> 40\%$	Maximize panel	On release
Two-hand squeeze	Both hands pinching, distance shrinks $> 40\%$	Restore to normal size	On release
Swipe left / right	Wrist x-velocity > 2 h/s, mostly horizontal	Cycle focus between panels	Cooldown 600 ms
Swipe down (open palm)	Armed; open palm moving down > 2 h/s, mostly vertical	Close focused panel	Cooldown 800 ms
Thumbs up	Four fingers curled; thumb extended (tip ≥ 0.6 h from the index knuckle) and pointing up, within 35° of vertical	Confirm pending action	Hold 600 ms
Thumbs down	Same shape with the thumb pointing down	Cancel pending action	Hold 600 ms

7.1 Arbitration rules (look-alike gestures)

- **One gesture at a time.** Two-hand gestures outrank one-hand gestures, hold gestures outrank swipes, and Point is the default when nothing else matches.
- **Pinch-drag vs two-hand spread/squeeze.** When one hand starts pinching, the recognizer waits up to 200 ms for the other hand. If both pinch, it is a two-hand gesture; if not, a drag starts at 300 ms. If the second hand pinches during a drag, the drag hands over to a two-hand resize of the same panel without dropping it.
- **Fist vs thumbs up/down.** The thumb decides: tucked means fist, extended and pointing up or down means thumbs. If the shape changes during a hold, the hold timer restarts, so a fist that opens into a thumbs-up never fires both.
- **Arm vs swipe down.** Arm needs a still palm and only fires while disarmed; Swipe down needs a fast downward palm and only fires while armed. Once armed, a still open palm just resets the auto-disarm timer.
- **Swipes after a drag.** Swipes are ignored during a pinch or drag and for 300 ms after release, so letting go of a panel can't flick it away.

7.2 Reliability rules

- Every gesture needs to be detected for N consecutive frames (default 4) before firing.
- Show a visual "charging" ring on the cursor while a hold gesture builds, so you see it coming.
- Ignore gestures when tracking confidence is below 0.6.

- The settings panel exposes every threshold so you can tune them live without editing code.
- Record landmark sessions to JSON so gestures, including each look-alike pair, can be replayed in unit tests.

8. WebSocket Protocol

All messages are JSON: { "type": string, "payload": object, "id": string }.

8.1 Connection and authentication

- The core binds to 127.0.0.1 only and rejects connections whose Origin header isn't the HUD's own (http://localhost:5173 in development, the core's own address when it serves the built HUD). This stops other websites open in your browser from connecting.
- Browsers can't add custom headers to a WebSocket, and tokens in a URL end up in logs, so the token travels in the first message. The HUD must send auth within 2 seconds of connecting; otherwise, or if the token is wrong, the core closes the connection with code 4401.
- The setup script generates the token and writes it to backend/.env (JARVIS_TOKEN) and frontend/.env.local (VITE_JARVIS_TOKEN). Both files are gitignored.

8.2 Messages

Direction	type	payload example
HUD -> Core	auth	{ "token": "..." } (must be the first message)
Core -> HUD	auth_ok	{ "session": "s1" }
HUD -> Core	user_text	{ "text": "open my calendar" }
HUD -> Core	gesture_event	{ "gesture": "thumbs_up", "panelId": "p12" }
HUD -> Core	confirm	{ "actionId": "a7", "approved": true }
Core -> HUD	state	{ "state": "listening" "thinking" "speaking" "idle" }
Core -> HUD	transcript	{ "role": "user" "jarvis", "text": "..." }
Core -> HUD	show_panel	{ "panel": { Panel object } }
Core -> HUD	update_panel / close_panel	{ "panelId": "p12", "data": {...} }
Core -> HUD	confirm_request	{ "actionId": "a7", "summary": "Send email to ...?" }
Core -> HUD	provider	{ "active": "claude_plan", "paid": false, "coolingDown": ["codex_plan"] }
Core -> HUD	error	{ "message": "..." }

9. Project Structure

```

jarvis/
  backend/
    main.py          config.py          events.py
    brain/           router.py         budget.py      prompt.py
                  providers/       claude_cli.py  codex_cli.py
                                gemini_free.py  anthropic_api.py
    tools/           registry.py      mcp_server.py  calendar.py   email.py
                  files.py         web.py         weather.py    system.py     notes.py
    auth/           google.py
    memory/         store.py         embeddings.py
    voice/          wake.py         stt.py         tts.py
    tests/          fixtures/      (recorded CLI output, landmark sessions)
    pyproject.toml  config.toml          .env.example
  frontend/
    index.html      vite.config.js      package.json    .env.example
    src/
      main.js
      hud/          scene.js         panel.js        effects.js      orb.js          statusbar.js
      gestures/     tracker.js      recognizer.js   arbiter.js     controller.js   filters.js
      net/          socket.js
      styles/       hud.css
      public/models/ (MediaPipe hand_landmarker.task)
    scripts/        setup.sh      (venv, installs, token)  start.sh
    docs/           README.md    gestures.md    protocol.md    providers.md

```

10. Build Roadmap

Timeline assumes roughly 8-10 hours per week. Each phase ends with something you can demo.

Phase	Scope	Done when...	Est.
0. Setup	Repo, folders, venv, Vite app, .env files and token, README; check c laude and codex are signed in to your plans	Both apps start with one command each	1-2 days
1. Brain	Terminal chat through c laude -p, tool registry + MCP server, 3 tools (time, web search, notes)	A question that needs a tool gets a correct answer, with \$0 API spend	Week 1
2. HUD + gestures	Standalone page: scene, 1 panel, cursor, arm, pinch-drag, fist, spread, arbiter	You can move/minimize/maximize a panel by hand smoothly; look-alike recordings pass	Week 2
3. Connect	WebSocket link with Origin check and first-message auth, transcript strip, show_panel, orb states, text input, provider badge	Typing in the HUD triggers tools and opens panels	Week 3
4. Real tools	Calendar + Gmail (production OAuth + reconnect), files, weather, system, confirm flow with thumbs up	Daily-use tasks work end to end with confirmations	Week 4
5. Fallback chain	Codex provider, local Ollama model, paid API with budget cap, failover tests	Forcing each provider "at limit" moves the same request to the next one, and paid spend stops at the cap	Week 5
6. Memory	Facts, history, summaries, local embeddings, remember/forget	JARVIS recalls something from a previous session	Week 6
7. Voice	Wake word, STT, TTS, barge-in (stop speaking when you talk)	Full hands-free voice conversation works	Week 7
8. Polish	More gestures, sounds, settings panel, startup script, more panel types	You'd happily leave it running all day	Week 8+

11. Non-functional Requirements

Area	Target
Gesture latency	< 100 ms from hand movement to on-screen response
Tracking frame rate	>= 25 fps on a mid-range laptop webcam
Voice response	< 3 s for simple requests on the primary path (STT + warm Claude CLI + start of TTS); fallback tiers may take longer
Failover	Switching provider adds < 2 s to the affected request
Wake word	< 1 false trigger per hour of normal room noise
Cost	\$0 beyond your existing subscriptions in a normal month; paid spend never exceeds the monthly cap
Resource use	Core idle < 500 MB RAM (not counting the optional local model); HUD stays smooth at 60 fps with 6 panels
Reliability	Auto-reconnect WebSocket; any tool or provider failure returns a spoken, friendly error

12. Security & Privacy

- Webcam frames are processed in the browser only; no video is stored or sent anywhere.
- API keys and OAuth tokens live in .env or ~/.jarvis/ token files, never committed (.gitignore).
- WebSocket bound to 127.0.0.1 with an Origin check and a token sent as the first message (section 8.1). The MCP server is also local-only and requires its own token.
- The subscription CLIs are locked down: Claude Code gets only the JARVIS MCP server plus web search and fetch; Codex runs with its shell tool disabled, in a read-only sandbox and an empty working folder. Neither can run commands or edit files except through JARVIS tools.
- JARVIS never reads, copies or reuses the CLIs' login tokens.
- Risky tools (send, delete, write, purchase) always require explicit voice or thumbs-up confirmation, enforced inside JARVIS's tool runner for every provider.
- Tool outputs from web pages and emails are treated as data, never as instructions (prompt-injection guard in the system prompt).
- Free cloud tiers that may train on prompts never receive email, file or memory content.
- Log file records tool calls, the provider used and the cost of each turn, with a size cap and easy clear option.

13. Hardware & Running Costs

- **Minimum:** laptop or desktop with a 720p webcam, mic and speakers, 8 GB RAM.
- **Recommended:** 1080p webcam, decent USB mic or speakerphone, 16 GB RAM (enough for an 8B-class local model in Ollama), large monitor or TV for the HUD.
- **Later / always-on:** mini PC near the display; the HUD can run on any browser on the network.
- **Accounts and software:** Claude Pro or Max with Claude Code installed and signed in; ChatGPT Plus or Pro with the Codex CLI signed in; Ollama (optional); a Google account. An Anthropic API key is only needed from Phase 5.
- **Costs:** the subscriptions you already pay for cover the brain in a normal month. Speech, wake word, voice, the local model, embeddings, Open-Meteo, the Google APIs and Tavily's free plan cost nothing. The paid Claude API runs only when all of these are unavailable, and never above the monthly cap (default US\$10).

14. Testing Plan

- **Unit tests (pytest):** tool registry schema generation, the MCP server exposing the same tools, each tool with mocked APIs, memory save/recall.
- **Router tests:** replay recorded CLI output (normal reply, usage limit, signed out, timeout, unreadable output) and check the right next provider, cooldown time and no repeated tool calls.
- **Budget tests:** simulated token usage crosses the 80% warning and the 100% cutoff; ask and off modes behave as specified.
- **Brain tests:** a set of 20 sample prompts with expected tool calls, run on the subscription providers after every prompt change. Test runs never use the paid API.
- **Gesture tests:** replay recorded landmark JSON through recognizer.js and arbiter.js and assert the expected events fire (and nothing else), with a recording for each look-alike pair in 7.1.
- **Protocol tests:** a wrong Origin and a missing, late or wrong token are all rejected.
- **Manual checklist:** different lighting, sitting vs standing, one vs two hands, someone walking behind you.
- **Latency logging:** timestamp each stage (wake, STT, provider, tool, TTS) and record which provider answered.

15. Risks & Mitigations

Risk	Mitigation
Accidental gesture triggers	Arm gesture, hold times, consecutive-frame checks, cooldowns, visible charging ring
Look-alike gestures firing together	Arbitration rules (7.1) and replay tests for each pair
Poor tracking in low light	Recommend a lamp facing you; show a tracking-quality indicator
Gorilla arm fatigue	Voice stays the primary control; gestures for spatial actions only
Subscription limits reached	Automatic failover down the provider chain; JARVIS owns the history so context carries over
Subscription CLI too slow for voice	Warm process, short custom system prompt, stream the first sentence to TTS; measure in Phase 1
Provider terms or CLI output change	One adapter per provider, contract tests on recorded output, re-check terms when providers announce changes
Unexpected paid spend	Paid tier last, monthly cap in JARVIS and in the Anthropic Console, overage off on both plans, PAID badge
Google sign-in expiring	Production publishing status; Reconnect panel when a refresh fails
Scope creep	Stick to the phase list; park new ideas in a backlog file

16. Backlog (after v1)

- Smart-home control (Home Assistant API).
- Face recognition to greet you and only arm gestures for you.
- Screen-aware help: screenshot -> "what's this error?".
- Phone companion page to send commands to the HUD from anywhere on your network.
- Daily briefing panel each morning (weather, calendar, emails, news).
- Hand-drawn gestures (draw a circle to open the calendar).
- WebXR mode for headsets later on.

17. First Steps Checklist

- Create the GitHub repo and folder structure from section 9.
- Check both CLIs are signed in to your plans (run `c laude once`, and `codex login status`), and turn off pay-as-you-go overage on both plans.
- Build Phase 1: terminal chat through `c laude -p + @tool registry + MCP server + 3 tools`.
- In parallel, spin up the Vite app and get MediaPipe drawing landmarks on screen from your webcam.
- Implement pinch-drag on one panel, then demo it to yourself before moving on.
- Leave the paid API until Phase 5. Then create an Anthropic API key and set a Console spend limit equal to `paid_monthly_cap_usd`.

Revision History

Version	Date	Changes
1.0	3 Oct 2026	First version.
1.1	5 Oct 2026	Subscription-first model router with free and capped paid fallback (section 3). Arbitration rules for look-alike gestures (7.1). WebSocket auth via first message plus Origin check (8.1). Google sign-in kept alive with production status and a reconnect flow (5.5). Free services named for weather, search and embeddings. Layout fixes.